
ShieldFi Protocol

Architecture & Design Document

Blockchain Technologies 2 — Final Project
Option E — Decentralized Insurance Pool

1. Executive Summary

ShieldFi is a fully on-chain decentralized parametric insurance protocol deployed on an Ethereum Layer 2 network. The protocol enables users to purchase coverage against oracle-verified price events. Underwriters deposit collateral into an ERC-4626 vault, earning yield from premiums while bearing payout risk. Claims are settled automatically by comparing a Chainlink price feed reading against a pre-agreed trigger price.

The system is governed by a DAO comprising token holders who vote on parameter changes such as premium rates, treasury fee percentages, and acceptable risk types. All governance actions are subject to a two-day Timelock delay. The protocol is composed of nine production contracts, a subgraph for off-chain indexing, and a React-based frontend dApp.

Key design goals:

- Non-custodial: user funds only leave the vault through verified claims or user-initiated withdrawals.
 - Trustless settlement: claim payouts are triggered by Chainlink oracle data, not by an admin.
 - Upgradeable: all core contracts use the UUPS proxy pattern, with upgrade authority gated to the Timelock.
 - Auditable: full event emission, The Graph indexing, and on-chain state readable by the verification script.
-

2. System Context Diagram (C4 Level 1)

The following describes the high-level actors and external systems that interact with ShieldFi.

External Actors:

- Policyholder — buys parametric insurance policies via the frontend dApp or directly on-chain.
- Underwriter — deposits stablecoins into the ERC-4626 vault to earn premium yield.
- Governor — any token holder who proposes or votes on protocol governance actions.
- Keeper / Anyone — triggers permissionless execution of queued Timelock operations after delay.

External Systems:

- Chainlink Price Feeds — provides tamper-resistant price data used for claim settlement and staleness validation.
- The Graph — indexes ShieldFi events, serving indexed data to the frontend via GraphQL queries.
- L2 Network (Arbitrum / Optimism / Base / zkSync Sepolia) — low-cost settlement layer where all contracts are deployed and verified.
- OpenZeppelin Contracts — battle-tested base implementations for ERC-20, ERC-4626, UUPS, AccessControl, Governor, and Timelock.

C4 Level 1 — Context:

Actor / System	Interaction with ShieldFi	Trust Level
Policyholder	buyPolicy(), fileClaim() via PolicyManager	Untrusted user
Underwriter	deposit(), withdraw() via InsuranceVault	Untrusted user
Governor / Token Holder	propose(), castVote(), queue(), execute() via Governor + Timelock	Token-weighted
Chainlink Oracle	Returns price data consumed by ClaimProcessor	Trusted external feed
The Graph Node	Receives events, exposes GraphQL API	Read-only observer
L2 Network	Executes transactions, stores contract state	Consensus layer

3. Container and Component Diagram (C4 Level 2)

The ShieldFi system is composed of nine on-chain contracts, a subgraph, and a frontend application. The diagram below describes each container and its primary responsibilities.

3.1 Smart Contract Layer

Contract	Role	Pattern
InsuranceVault (UUPS Proxy)	ERC-4626 vault holding underwriter collateral; pays out claims; earns premium yield	UUPS, ERC-4626, AccessControl, Pausable, ReentrancyGuard
InsuranceVaultV2	V2 upgrade demonstrating storage-safe extension of InsuranceVault	UUPS upgrade path
PolicyManager (UUPS Proxy)	Registers policies, collects premiums, routes fees to treasury, coordinates claim settlement	UUPS, AccessControl, CEI, ReentrancyGuard
ClaimProcessor	Consults Chainlink feed, validates staleness, triggers vault payout	Oracle adapter, CEI
ShieldProtocolGovernor	OZ Governor with ERC20Votes, quorum, voting period, and Timelock binding	Governor, GovernorTimelockControl
TimelockController	2-day delay enforcer; controls treasury withdrawals and parameter upgrades	Timelock
ShieldGovToken	ERC-20 governance token with ERC20Votes + ERC20Permit	ERC-20, ERC20Votes
PolicyNFT	ERC-721 NFT minted per policy for policy-holder UX and transferability	ERC-721
ProtocolTreasury	Holds accumulated protocol fees; withdrawals restricted to Timelock only	Access Control
PoolFactory	Deploys PolicyPool instances via CREATE and CREATE2	Factory
GasOptimizedMath (library)	Inline Yul vs Solidity mulDiv benchmark	Yul assembly

3.2 Access Control Roles

Role	Holder at Deploy	Capabilities
DEFAULT_ADMIN_ROLE	Deployer (then transferred to Timelock)	Grant / revoke any role
UPGRADER_ROLE	Timelock	Authorize UUPS upgrades
PAUSER_ROLE	Deployer multisig	Pause / unpause contracts
CLAIM_PAYER_ROLE	ClaimProcessor	Call vault.payClaim()
MINTER_ROLE (NFT)	PolicyManager	Mint PolicyNFT on purchase

Role	Holder at Deploy	Capabilities
EXECUTOR_ROLE (Timelock)	address(0) — permissionless	Execute queued operations
PROPOSER_ROLE (Timelock)	Governor	Queue operations after vote

3.3 Proxy Layout

InsuranceVault and PolicyManager are deployed behind ERC-1967 transparent-compatible UUPS proxies. The implementation address is stored at slot EIP-1967.implementation. Upgrade authority is held exclusively by the UPGRADER_ROLE, which is granted to the Timelock after deployment, ensuring all upgrades pass a DAO vote and 2-day delay.

- Proxy slot: `bytes32(uint256(keccak256('eip1967.proxy.implementation')) - 1)`
 - InsuranceVaultV2 extends V1's storage layout with a new field placed after the `__gap` array.
 - Storage collision risk: mitigated by the 50-slot `__gap` in V1 and OpenZeppelin's upgradeable base contracts.
-

4. Sequence Diagrams — Critical User Flows

4.1 Policy Purchase Flow

Step	Actor	Action	Contract
1	User	Approves premium amount on stablecoin	ERC-20 (USDC)
2	User	Calls buyPolicy(coverage, triggerPrice, duration, triggerAbove)	PolicyManager
3	PolicyManager	Computes premium via GasOptimizedMath.premiumMulDivYul	GasOptimizedMath
4	PolicyManager	Transfers premium from user via SafeERC20.safeTransferFrom	ERC-20
5	PolicyManager	Splits fee to ProtocolTreasury (treasuryFeeBps), remainder to InsuranceVault	ProtocolTreasury + InsuranceVault
6	PolicyManager	Stores Policy struct; increments nextPolicyId	PolicyManager storage
7	PolicyManager	Calls PolicyNFT.mint(user, policyId) — requires MINTER_ROLE	PolicyNFT
8	PolicyManager	Emits PolicyPurchased event (indexed by subgraph)	The Graph

4.2 Claim Settlement Flow

Step	Actor	Action	Contract
1	Policyholder	Calls processClaim(policyId)	ClaimProcessor
2	ClaimProcessor	Fetches latestRoundData() from Chainlink feed	AggregatorV3Interface
3	ClaimProcessor	Validates: answer > 0, updatedAt > block.timestamp - heartbeat, answeredInRound >= roundId	ClaimProcessor
4	ClaimProcessor	Reads Policy from PolicyManager; checks expiry, active, not claimed	PolicyManager
5	ClaimProcessor	Evaluates trigger: price >= triggerPrice (if triggerAbove) or price <= triggerPrice	ClaimProcessor
6	ClaimProcessor	Marks policy claimed via PolicyManager.markClaimed(policyId)	PolicyManager
7	ClaimProcessor	Calls InsuranceVault.payClaim(beneficiary, amount) — requires CLAIM_PAYER_ROLE	InsuranceVault
8	InsuranceVault	CEI: updates totalPayouts before safeTransfer; emits ClaimPayout	InsuranceVault

4.3 Governance Propose → Vote → Queue → Execute Flow

Step	Actor	Action
1	Proposer (min 1% of supply)	Calls <code>governor.propose([targets],[values],[calldatas], description)</code>
2	Governor	Validates <code>proposalThreshold</code> ; emits <code>ProposalCreated</code> ; enters Pending state
3	All holders	After 1-day voting delay, proposal becomes Active
4	Voters	Call <code>governor.castVote(proposalId, support)</code> — votes weighted by <code>ERC20Votes</code> checkpoints
5	Governor	After 1-week voting period, evaluates quorum (4%) and For/Against; state → Succeeded or Defeated
6	Anyone	Calls <code>governor.queue(proposalId)</code> — schedules operation in <code>TimelockController</code>
7	<code>TimelockController</code>	Enforces <code>minDelay = 2 days</code> ; operation enters Queued state
8	Anyone	After delay, calls <code>governor.execute(proposalId)</code> — <code>Timelock</code> executes the <code>calldata</code>
9	Target Contract	Receives the governance-approved call (e.g., <code>updateTreasuryFeeBps</code> , <code>upgradeTo</code>)

5. Data Model — Storage Layouts

5.1 InsuranceVault Storage

Slot (approx.)	Variable	Type	Notes
Inherited OZ slots	ERC20 / ERC4626 / AC / Pausable / RG / UUPS	Various	Inherited from OZ upgradeable base contracts
Custom slot 0	totalPayouts	uint128	Cumulative payout counter
Custom slots 1-50	__gap	uint256[50]	Reserved for future V2 storage additions

5.2 PolicyManager Storage

Variable	Type	Description
asset	IERC20	Stablecoin used for premiums and payouts
vault	InsuranceVault	Reference to the ERC-4626 vault
treasury	ProtocolTreasury	Reference to the protocol treasury
nextPolicyId	uint256	Auto-incrementing policy ID counter
treasuryFeeBps	uint16	Fee percentage routed to treasury (basis points)
annualPremiumBps	uint16	Annual premium rate in basis points
claimProcessor	address	Address authorized to mark claims
_policies	mapping(uint256 => Policy)	All policy data keyed by policyId

5.3 Policy Struct

Field	Type	Description
buyer	address	Policyholder address
purchasedAt	uint48	Block timestamp at purchase
expiresAt	uint48	Block timestamp at expiry
triggerAbove	bool	True if payout triggers when price >= triggerPrice
claimed	bool	Whether claim has been settled
active	bool	Whether policy is active
coverageAmount	uint256	Maximum payout in asset decimals
premiumPaid	uint256	Premium collected at purchase
triggerPrice1e8	int256	Oracle price threshold (8 decimal places, Chainlink format)

5.4 Storage Collision Analysis for UUPS Upgrades

InsuranceVaultV2 extends V1 by appending new storage variables after the inherited `__gap`. The OZ upgradeable base contracts guarantee no overlap with base class storage. The `__gap` array consumes slots that V2 fills from slot 0 of the gap, maintaining layout compatibility.

- V1: [...OZ slots..., totalPayouts (uint128), `__gap`[50]]
- V2: [...OZ slots..., totalPayouts (uint128), newField (using first gap slot), `__gap`[49]]
- Verification: forge test runs a fork upgrade test asserting that V2 reads V1 state correctly.

6. Trust Assumptions & Threat Model

6.1 Role Trust Levels

Principal	What They Can Do	Risk if Compromised
Timelock (2-day delay)	Upgrade contracts, change parameters, withdraw treasury funds	Full protocol control — mitigated by 2-day delay allowing emergency response
Governor (DAO)	Propose and queue any Timelock action after vote	Governance attacks possible — mitigated by quorum, proposal threshold, and Timelock
PAUSER_ROLE holder	Pause all state-changing functions	DOS risk — mitigated by Timelock controlling un-pause in worst case
ClaimProcessor	Trigger vault payouts	Could drain vault if compromised — mitigated by per-claim policy checks
Chainlink Feed	Provides price data for claim settlement	Stale / manipulated prices rejected by staleness and sanity checks

6.2 What the Protocol Cannot Protect Against

- A supermajority governance attack (>50% of supply hostile): mitigated by proposal threshold and quorum but not fully preventable.
- L2 sequencer censorship: users can force-include transactions via the L1 bridge in extremis.
- Chainlink oracle going offline permanently: claims freeze until a new feed is set via governance.

7. Design Patterns — Justified Usage

Pattern	Where Used	Justification
UUPS Proxy (Upgradeability)	InsuranceVault, PolicyManager	Required for bug fixes and parameter evolution without breaking deployed addresses.
Factory (CREATE + CREATE2)	PoolFactory → PolicyPool	Allows deterministic address prediction for cross-chain deployments; CREATE2 enables counterfactual deployments.
Checks-Effects-Interactions (CEI)	InsuranceVault.payClaim(), ClaimProcessor.processClaim()	Prevents reentrancy by updating state (counters, flags) before external calls.
ReentrancyGuard	InsuranceVault, PolicyManager, ShieldAMM	Defense-in-depth against reentrancy; supplements CEI in all functions with external token transfers.
Access Control / RBAC	All production contracts	Fine-grained permission model prevents privilege escalation; roles documented in Section 3.2.
Pausable / Circuit Breaker	InsuranceVault, PolicyManager	Emergency stop controlled by PAUSER_ROLE to halt deposits and claims during incidents.
Oracle Adapter / Interface Abstraction	ClaimProcessor via AggregatorV3Interface	Decouples claim logic from feed implementation; allows substituting a mock in tests.
Timelock (Governance Delay)	TimelockController (2-day minDelay)	Ensures all privileged parameter changes have a 48-hour response window for the community.
Pull-over-Push Payments	InsuranceVault withdrawal, ShieldAMM removeLiquidity	Users initiate withdrawals rather than contracts pushing funds, eliminating failed-send griefing.
State Machine (Policy Lifecycle)	PolicyManager / ClaimProcessor	Policy transitions: Active → Expired Claimed, preventing double-claims and out-of-sequence operations.

8. Architecture Decision Records (ADR)

ADR-01: ERC-4626 for Underwriter Vault

Field	Detail
Context	Underwriters need a standardized, composable way to deposit collateral and receive yield-bearing shares.
Options Considered	Custom vault with manual accounting; ERC-4626 standard vault
Decision	ERC-4626 via OZ ERC4626Upgradeable
Consequences	Standard interface allows integration with any ERC-4626-compatible protocol; rounding invariants tested by forge fuzz suite.

ADR-02: UUPS over Transparent Proxy

Field	Detail
Context	Both proxy patterns offer upgradeability; choice impacts gas and admin key management.
Options Considered	TransparentUpgradeableProxy (OZ), UUPS
Decision	UUPS — upgrade logic in the implementation contract, not the proxy.
Consequences	Lower deployment gas; upgrade gate enforced by UPGRADER_ROLE on the implementation rather than a separate ProxyAdmin contract.

ADR-03: Parametric vs Indemnity Insurance

Field	Detail
Context	Indemnity insurance requires loss verification; parametric triggers on a measurable event.
Options Considered	Indemnity model (requires claims adjuster oracle), parametric model (Chainlink price trigger)
Decision	Parametric model keyed on Chainlink price threshold.
Consequences	Fully trustless settlement; no subjective adjudication; limited to price-event coverage types.

ADR-04: Governor Block-Based vs Timestamp-Based Voting

Field	Detail
Context	OZ Governor supports both clock modes; L2 blocks may vary in timing.
Options Considered	Block-number clock (default), timestamp clock (ERC-6372)
Decision	Block-number clock (default OZ behavior).
Consequences	Simpler; L2 block times are consistent enough for the capstone scope. Production would use timestamp clock for cross-chain compatibility.

9. Layer 2 Deployment & Gas Analysis

9.1 Target Network

All contracts are deployed and verified on an Ethereum Layer 2 testnet (Arbitrum Sepolia, Optimism Sepolia, Base Sepolia, or zkSync Sepolia). The deployment script (script/DeployShieldFi.s.sol) is idempotent and parameterized; the verify script (script/VerifyShieldFi.s.sol) checks ownership, Timelock delay, and Governor parameters post-deployment.

9.2 Gas Comparison Table — L1 vs L2

Operation	Estimated L1 Gas (Mainnet)	Estimated L2 Gas Cost	Savings Ratio
InsuranceVault.deposit()	~85,000 gas / ~\$8.50	~85,000 gas / ~\$0.05	~170x
PolicyManager.buyPolicy()	~120,000 gas / ~\$12.00	~120,000 gas / ~\$0.07	~170x
ClaimProcessor.processClaim()	~95,000 gas / ~\$9.50	~95,000 gas / ~\$0.06	~160x
ShieldAMM.swap()	~70,000 gas / ~\$7.00	~70,000 gas / ~\$0.04	~175x
Governor.castVote()	~60,000 gas / ~\$6.00	~60,000 gas / ~\$0.04	~150x
Timelock.execute()	~55,000 gas / ~\$5.50	~55,000 gas / ~\$0.03	~180x

Note: L2 gas costs assume ~\$0.0006/gas on L2 testnet vs ~\$0.0001/gas on mainnet with 10 gwei base fee. Production numbers may vary.

10. Subgraph & Frontend Architecture

10.1 Subgraph Entities

Entity	Source Event	Key Fields
VaultDeposit	ERC4626 Deposit	vault, sender, owner, assets, shares, timestamp
VaultWithdraw	ERC4626 Withdraw	vault, sender, receiver, owner, assets, shares, timestamp
PolicyPurchase	PolicyManager.PolicyPurchased	policyId, buyer, coverage, premium, expiration, timestamp
ClaimProcessed	ClaimProcessor.ClaimSettled	policyId, beneficiary, payout, oraclePrice, timestamp
TreasuryWithdraw	ProtocolTreasury.Withdrawn	treasury, token, to, amount, timestamp

10.2 Documented GraphQL Queries

- RecentDeposits — last N vault deposits ordered by timestamp desc.
- RecentWithdraws — last N vault withdrawals ordered by timestamp desc.
- RecentPolicies — last N policy purchases with buyer, coverage, premium, expiration.
- RecentClaims — last N settled claims with payout and oracle price.
- RecentTreasuryMoves — last N treasury withdrawals showing fee flows.

10.3 Frontend dApp Architecture

- Wallet connection: MetaMask (primary) + WalletConnect (secondary connector).
 - Read state: token balance, voting power, delegate address, vault share balance, pool reserves.
 - Write transactions: deposit to vault, buy policy, cast governance vote (3 state-changing UI flows).
 - Governance panel: active proposal list with state (Pending / Active / Succeeded / Defeated / Queued / Executed) and vote button.
 - Subgraph integration: Recent Policies and Recent Claims sections read from The Graph, not from RPC calls.
 - Error handling: network mismatch prompt, insufficient balance message, transaction rejection toast — no raw RPC errors exposed.
-

11. DevOps & CI Pipeline

The GitHub Actions workflow (`.github/workflows/shieldfi-ci.yml`) runs on every push and pull request:

- Step 1 — Install Foundry toolchain and Node.js dependencies.
- Step 2 — forge build: compile all contracts including tests.
- Step 3 — forge test --fork-url \$ARB_SEPOLIA_RPC: run full test suite including fork tests.
- Step 4 — forge coverage: generate line coverage report; fail if below 90%.
- Step 5 — Slither (crytic/slither-action): static analysis gated to fail on High findings.
- Step 6 — forge fmt --check + solhint + Prettier: lint checks for Solidity and frontend code.

A PR cannot be merged while CI is red. The coverage report is committed to the repo as `coverage/report.md`.
